부산대학교
PUSAN NATIONAL UNIVERSITY

AVL Rotations

Standing the Tree Back Up

Single and Double Rotations

DS3600082-140
자료구조
2026학년도 2학기

오용경
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Section 1 • AVL Rotations

What a Rotation Preserves

회전이 무엇을 바꾸고 무엇을 지키는지 정한다

기울어진 트리를 다시 세우려면

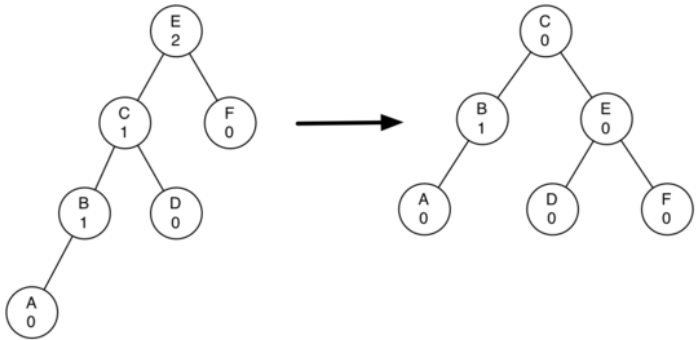
정의

회전(rotation)

부모와 자식 노드의 관계를 국소적으로 바꾸되 중위 순서를 보존하는 연산이다.
참조 몇 개만 옮기므로 담긴 값의 수와 상관없이 $O(1)$이다.

AVL 트리의 재균형(rebalance)은 회전으로 불균형 부분 트리의 높이를 삽입 전 수준으로 되돌린다

오른쪽 회전 한 번

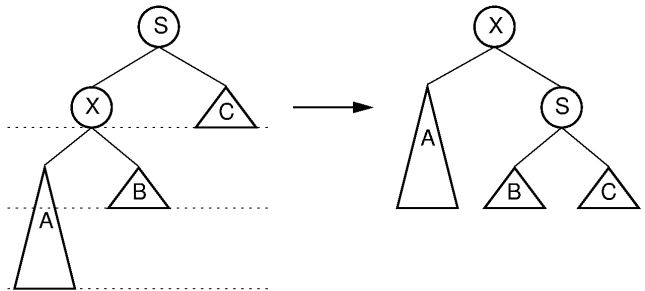


노드 안의 아래 숫자가 균형 인수다. 회전 전에는 E가 2였고, 회전 뒤에는 모든 노드가 0 또는 1이다.

Source: Miller and Ranum (2013), AVL Tree Implementation.

회전은 이진 탐색 트리의 순서를 보존한다

부분 트리 *A*, *B*, *C*의 왼쪽에서 오른쪽 순서는 회전 전후에도 같다



$A < X < B < S < C$의 순서가 그대로이므로 회전 뒤에도 이진 탐색 트리 조건을 만족한다.

Source: OpenDSA (n.d.), AVL Tree Rotations.

왼쪽 회전과 오른쪽 회전

항목	왼쪽 회전	오른쪽 회전
쓰는 경우	오른쪽이 무거울 때	왼쪽이 무거울 때
올라가는 노드	오른쪽 자식 노드	왼쪽 자식 노드
코드 이름	`rotate_left`	`rotate_right`

두 회전은 서로의 거울상이므로 방향만 바꾸면 절차가 같다

회전에서 바꾸는 노드와 트리의 참조

필드	저장하는 것
`key`, `value`	탐색할 키와 그 키에 대응하는 값
`left_child`, `right_child`	왼쪽·오른쪽 자식 참조, 없으면 `None`
`parent`	부모 참조, 전체 트리의 루트에서는 `None`
`balance_factor`	저장된 $h(\text{left}) - h(\text{right})$
트리의 `_root`	트리 객체가 저장하는 전체 루트 참조

자식 참조는 BST의 `left`/`right`와 같은 역할이다. 자식의 `parent`도 함께 고쳐야 부모를 따라 올라갈 수 있다.

Adapted from Miller and Ranum (2013), Search Tree Implementation and AVL Tree Implementation.

회전 전에 부모와 루트의 연결을 구분한다

메서드	확인하거나 수행하는 일
`is_left_child()`	부모가 있고 부모의 왼쪽 자식이 자신인지 확인
`is_right_child()`	부모가 있고 부모의 오른쪽 자식이 자신인지 확인
`is_root()`	`parent`가 `None`인 전체 루트인지 확인
`rotate_left(node)`	오른쪽 자식을 새 부분 트리 루트로 올림
`rotate_right(node)`	왼쪽 자식을 새 부분 트리 루트로 올림
`rebalance(node)`	두 균형 인수의 부호로 한 번·두 번 회전 선택

전체 루트를 돌리면 `_root`를, 그 밖에는 부모의 자식 참조를 바꾼다. 회전의 참조 변경과 별도로 두 회전 노드의 `balance_factor`도 갱신해야 한다.

Adapted from Miller and Ranum (2013), Search Tree Implementation and AVL Tree Implementation.

`rotate_left` 코드와 참조 변경

```
1  def rotate_left(self, rotation_root):
2      new_root = rotation_root.right_child
3      rotation_root.right_child = new_root.left_child
4      if new_root.left_child:
5          new_root.left_child.parent = rotation_root
6      new_root.parent = rotation_root.parent
7      if rotation_root.is_root():
8          self._root = new_root
9      elif rotation_root.is_left_child():
10         rotation_root.parent.left_child = new_root
11     else:
12         rotation_root.parent.right_child = new_root
13     new_root.left_child = rotation_root
14     rotation_root.parent = new_root
```

줄	그림에서 바뀌는 것
1, 2	X가 축, S가 새 루트
3	X의 오른쪽 ← B
4, 5	B의 부모 ← X
6	S의 부모 ← X의 부모
7부터 12	위쪽 자식 참조 ← S
13	S의 왼쪽 ← X
14	X의 부모 ← S

그림은 (b)에서 (a)로 바뀐다. 코드의 `left_child`/`right_child`가 그림의 두 자식이다.

A와 C는 그대로 붙어 있고 참조 몇 개만 바뀌므로 n과 무관하게 $O(1)$이다

Adapted from Miller and Ranum (2013), AVL Tree Implementation.

Checkpoint

What a Rotation Preserves

- 회전이 바꾸는 것과 지키는 것은 각각 무엇인가?
- 회전의 비용이 $O(1)$인 이유는 무엇인가?
- 중위 순회 결과가 회전 뒤에도 같은 이유는 무엇인가?

Section 2 • AVL Rotations

Single Rotation

한 번의 회전으로 끝나는 경우를 구분한다

바깥으로 기운 경우

불균형 노드와 무거운 자식 노드가 같은 방향이면 single rotation으로 균형을 회복한다. 회전은 한 번만 필요하다.

경우	모양	할 일
left-left	왼쪽 자식의 왼쪽이 높다	오른쪽 회전 1회
right-right	오른쪽 자식의 오른쪽이 높다	왼쪽 회전 1회

$30, 20, 10$을 넣으면

단계	모양	판정
30 넣기	30	균형
20 넣기	30의 왼쪽에 20	균형
10 넣기	세 노드가 왼쪽으로 한 줄	30에서 $bf = +2$
오른쪽 회전 뒤	루트 20, 양쪽에 10과 30	세 노드 모두 $bf = 0$

30을 축으로 오른쪽 회전하면 20이 루트가 된다

불균형이 처음 생긴 노드를 축으로 돈다

항목	내용
회전의 축	$\lvert bf \rvert = 2$가 된 노드
그런 노드가 여럿일 때	새로 넣은 값에서 가장 가까운 노드
그 위쪽 조상	회전 뒤 높이가 돌아와 더 볼 필요가 없다

> 삽입에서는 회전이 한 자리에서만 일어난다

$10, 20, 30$을 넣으면

단계	모양	판정
10, 20 넣기	오른쪽으로 둘	균형
30 넣기	세 노드가 오른쪽으로 한 줄	10에서 $bf = -2$
왼쪽 회전 뒤	루트 20, 양쪽에 10과 30	세 노드 모두 $bf = 0$

10을 축으로 왼쪽 회전하면 20이 루트가 된다

두 균형 인수의 부호로 회전 종류를 정한다

불균형 노드와 그 무거운 자식 노드의 방향 두 개만 보면 된다

노드의 *bf*	자식 노드의 *bf*	판정
$+2$ (왼쪽이 무거움)	≥ 0	left-left, 바깥
$+2$	< 0	left-right, 안쪽
-2 (오른쪽이 무거움)	≤ 0	right-right, 바깥
-2	> 0	right-left, 안쪽

Checkpoint

Single Rotation

- left-left와 right-right 불균형에는 각각 어떤 회전이 필요한가?
- 회전의 축이 되는 노드는 어떻게 고르는가?
- 삽입 뒤 회전이 한 자리에서만 일어나는 이유는 무엇인가?

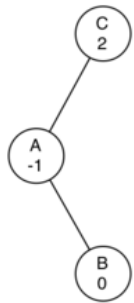
Section 3 • AVL Rotations

Double Rotation

회전 한 번으로 풀리지 않는 경우를 다룬다

안쪽으로 꺾이면 회전 한 번으로 풀리지 않는다

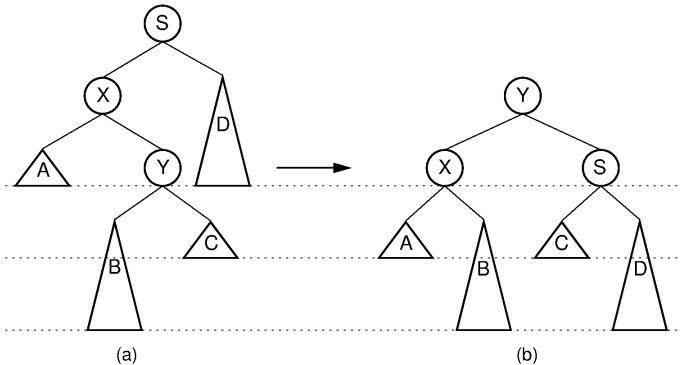
불균형 노드와 무거운 자식 노드가 반대 방향으로 기울어 있으면 한 번 돌려도 반대로 기운다



$A(-2)$의 오른쪽 자식 $C(+1)$가 왼쪽 자식 B를 가진 트리를 A에서 왼쪽으로 한 번 회전한 결과다. 이번에는 C가 $+2$가 되어(A는 -1) 여전히 불균형하다.

Source: Miller and Ranum (2013), AVL Tree Implementation.

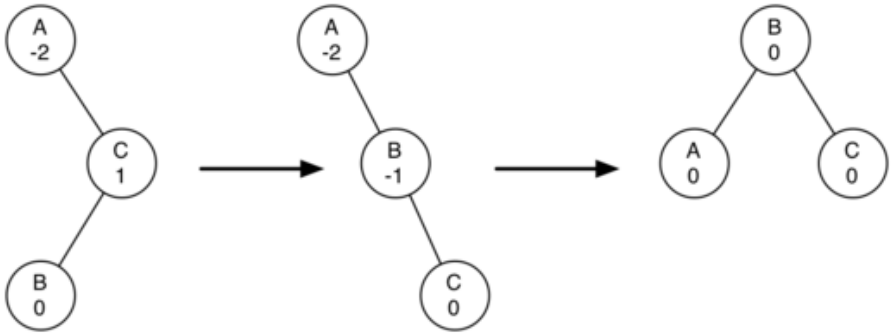
안쪽으로 꺾인 경우에는 두 번 회전한다



left-right 경우의 double rotation(자식 노드를 먼저 회전한 뒤 부모 노드를 회전하는 경우)이다. 삼각형 A부터 D는 부분 트리이고, X를 먼저 왼쪽으로, S를 오른쪽으로 회전한다.

Source: OpenDSA (n.d.), AVL Tree Rotations.

두 번에 나누어 돌리기



right-left 경우다. A=10, B=20, C=30으로 읽으면 10, 30, 20 삽입이다. 가운데는 C를 축으로 오른쪽 회전해 만든 right-right 바깥 경우다.

Source: Miller and Ranum (2013), AVL Tree Implementation.

안쪽 경우의 첫 회전은 중간 상태를 만든다

두 경우 모두 가운데 값 20을 위로 올린다

삽입 순서	첫 회전 뒤	둘째 회전 뒤
30, 10, 20	30의 왼쪽 20, 20의 왼쪽 10	루트 20, 양쪽에 10과 30
10, 30, 20	10의 오른쪽 20, 그 오른쪽 30	루트 20, 양쪽에 10과 30

첫 회전은 바깥 경우로 바꾸고, 둘째 회전은 높이 균형을 복구한다

Adapted from Miller and Ranum (2013), AVL Tree Implementation.

네 가지 경우

경우	종류	할 일
left-left	바깥	오른쪽 회전
right-right	바깥	왼쪽 회전
left-right	안쪽	왼쪽 뒤 오른쪽
right-left	안쪽	오른쪽 뒤 왼쪽

> 경우는 넷이지만 판정에 쓰는 것은 두 개의 방향뿐이다

Adapted from Morin (2013), Figure 9.10, and UBC DSCI 221 (n.d.), balanced trees.

두 회전의 축은 서로 다르다

순서	축	하는 일
첫 회전	무거운 자식 노드	안쪽을 바깥으로 낸다
둘째 회전	불균형 노드	바깥 경우처럼 세운다

Caution 두 회전을 같은 자리에서 반대 방향으로 하면 원래 모양으로 돌아오므로 두 회전의 축이 달라야 한다

Checkpoint

Double Rotation

- 안쪽 경우에서 회전 한 번이 실패하는 이유는 무엇인가?
- double rotation의 두 회전은 각각 어느 노드를 축으로 하는가?
- 네 가지 경우를 판정할 때 보는 두 방향은 무엇인가?

Section 4 • AVL Rotations

Insert, Update, Rotate

삽입에서 회전까지의 순서를 하나로 잇는다

삽입 뒤에 하는 일

값을 넣은 뒤 올라오면서 균형을 확인한다

단계	하는 일	어디에서
1	BST처럼 값을 넣는다	단말 노드 자리
2	올라가며 높이를 고친다	지나온 노드마다
3	$\lvert bf \rvert = 2$를 만나면 회전	처음 만난 곳
4	멈춘다	회전한 곳 위는 그대로

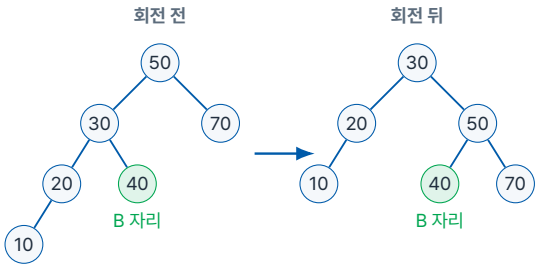
$50, 30, 70, 20, 40, 10$을 넣으면

넣은 값	일어난 일	그 뒤 루트
$50 \dots 40$	회전 없음	50
10	50에서 $bf = 2$	30

여섯 번의 삽입에 회전은 한 번뿐이었고 높이는 2로 유지되었다

50에서 오른쪽으로 돌린 뒤 다시 확인한다

30이 루트가 되고 왼쪽에 20, 오른쪽에 50이 놓인다. 30의 오른쪽에 있던 40은 50의 왼쪽으로 옮긴다



노드	회전 뒤 *bf*	남은 연결
20	$+1$	왼쪽 자식 10
50	0	왼쪽 40, 오른쪽 70
30	0	왼쪽 20, 오른쪽 50

중위 순서 $10 < 20 < 30 < 40 < 50 < 70$은 그대로이고 높이는 2로 돌아온다

Adapted from Miller and Ranum (2013), AVL Tree Implementation.

전체 비용

단계	비용	이유
자리 찾기	$O(\log n)$	높이만큼 내려간다
높이 갱신	$O(\log n)$	같은 길을 올라온다
회전	$O(1)$	최대 두 번
전체	$O(\log n)$	

균형을 지키는 추가 비용은 삽입 비용의 차수를 바꾸지 않는다

정렬된 값을 넣어 보면

1부터 7까지 차례로 넣는 최악의 순서를 시험한다

트리	높이	탐색 비용
균형을 잡지 않은 BST	6	$O(n)$
AVL 트리	2	$O(\log n)$

1부터 4까지 넣으며 회전 축을 찾는다

빈 트리에 값을 하나씩 넣고 새 노드에서 부모로 올라가며 첫 불균형 조상을 찾는다

삽입	축 (*bf*)	경우	삽입·재균형 뒤 전체 트리
1	없음	없음	1
2	없음	없음	$1(.,2)$
3	$1(-2)$	right-right	$2(1,3)$
4	없음	없음	$2(1,3(.,4))$

a(*b*, *c*)는 루트 *a*의 왼쪽·오른쪽 부분 트리가 *b*, *c*라는 뜻이다. 점은 빈 자리이며 단말 노드는 값만 적었다. 3 삽입 때 1에서 왼쪽 회전한다.

5부터 7까지 넣으며 회전 뒤 트리를 잇는다

4까지 넣은 트리에 이어 삽입한다. 세 경우 모두 표의 축에서 왼쪽으로 한 번 회전한다

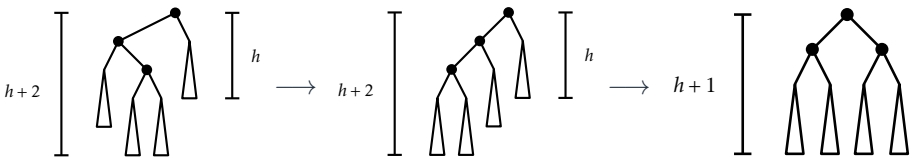
삽입	축 (*bf*)	경우	삽입·재균형 뒤 전체 트리
5	$3(-2)$	right-right	$2(1, 4(3, 5))$
6	$2(-2)$	right-right	$4(2(1, 3), 5(., 6))$
7	$5(-2)$	right-right	$4(2(1, 3), 6(5, 7))$

6 삽입의 회전에서 가운데 부분 트리 B인 노드 3은 4의 왼쪽에서 2의 오른쪽으로 옮겨지고, 3의 `parent`도 2로 바뀐다.

6 삽입 때 전체 루트가 2에서 4로 바뀌어 `_root`도 고친다. 5와 7 삽입 때에는 전체 루트가 유지되고 부모의 자식 참조만 새 부분 트리 루트에 잇는다

첫 불균형 조상의 회전으로 위쪽 높이도 복구된다

근거	내용
불균형이 생긴 원인 회전 뒤 부분 트리 높이 그 위 조상들의 높이	삽입으로 부분 트리 높이가 1 늘었다 삽입 전 수준으로 돌아간다 더 이상 바뀌지 않는다



left-right 경우다. 두 번 회전하면 $h+2$와 h로 기운 두 부분 트리가 모두 $h+1$이 되어 높이가 삽입 전과 같다. right-left는 좌우만 바뀐다.

삽입은 첫 불균형 조상에서 한 번의 재균형으로 끝난다

Adapted from Morin (2013), Figure 9.10.

Checkpoint

Insert, Update, Rotate

- 삽입 뒤 어떤 순서로 균형을 확인하는가?
- AVL 삽입 전체의 비용은 얼마이고 왜 그런가?
- 정렬된 값을 차례로 넣으면 BST와 AVL 트리는 어떻게 갈리는가?

Section 5 • AVL Rotations

Beyond AVL

균형을 지키는 다른 방법을 개념으로만 비교한다

레드-블랙 트리

정의

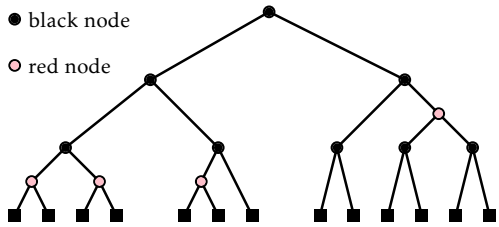
레드-블랙 트리(Red-Black tree)

각 노드에 빨강 또는 검정 색을 부여하고 색 규칙으로 트리의 높이를 제한하는 BST이다.
색 규칙은 둘이다. 루트에서 빈 자리(nil)까지 어느 경로든 검정 노드 수가 같고, 빨강 노드 두 개가 부모와 자식으로 이어지지 않는다.
AVL 트리처럼 높이를 $O(\log n)$으로 유지하지만 균형을 확인하는 기준이 다르다.

AVL 트리는 부분 트리의 높이 차이를, 레드-블랙 트리는 색 규칙을 이용해 높이를 제한한다

Source: Morin (2013), Section 9.2, RedBlackTree.

색 규칙을 지키는 레드-블랙 트리



채운 원은 검정, 속이 빈 원은 빨강 노드이고 사각형은 빈 자리(nil)다. 루트에서 어느 사각형까지 가도 검정 원은 3개이고 빨강끼리 이어진 곳은 없다.

> 두 규칙을 지키면 n개를 담은 레드-블랙 트리의 높이는 $2\log_2 n$ 이하다

Source: Morin (2013), Figure 9.4.

균형 규칙과 공통 보장을 함께 비교한다

필요한 연산 순서 기반 탐색·삽입·삭제

균형 규칙(둘 중 하나)

AVL 트리: 좌우 높이 차이

레드-블랙 트리: 색 규칙

공통 보장

최악의 경우 $O(\log n)$

두 구조는 높이를 제한하는 규칙이 다르지만 최악의 경우 $O(\log n)$을 함께 보장한다

두 방법의 차이

항목	AVL 트리	레드-블랙 트리
재는 것	높이 차이	색 규칙
균형의 정도	더 엄격하다	조금 느슨하다
최대 높이	약 $1.44 \log_2 n$	$2 \log_2 n$ 이하
탐색의 최악의 경우	$O(\log n)$	$O(\log n)$

AVL이 보통 더 낮은 높이를 유지하지만 실제 성능은 구현과 작업 부하에 따라 달라진다

질의 종류에 따라 균형 트리와 해시테이블을 고른다

필요한 질의	알맞은 구조
키 정렬 순회와 범위 질의	균형 트리가 적합하다
같은 키 조회(기대 $O(1)$)	해시테이블이 흔한 선택이다
Python `dict`	삽입 순서는 보존하지만 키 순서로 정렬하지는 않는다

> 자료구조는 필요한 순서와 질의 종류를 기준으로 고른다

균형 트리를 쓰는 이유

BST의 $O(n)$ 최악을 없애려고 회전이라는 대가를 치른 것이다

입력과 비용	BST	균형 트리
고르게 퍼지는 입력	$O(\log n)$	$O(\log n)$
한쪽으로 기우는 입력	$O(n)$	$O(\log n)$
치른 대가	없음	회전과 상태 저장

Checkpoint

Beyond AVL

- 레드-블랙 트리는 무엇으로 균형을 재는가?
- AVL 트리가 더 엄격하다는 것은 무슨 뜻인가?
- 키 범위 질의에 균형 트리가 적합한 이유는 무엇인가?

How Do Graphs Extend Trees?

Next Lecture: Graphs and Edges

- 정점과 간선
- 유향 그래프와 무향 그래프
- 경로와 도달 가능성

오용경 yongkyung.oh@pusan.ac.kr